

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №10

«Файловые потоки. Работа с файлами.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 15

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

1. Создать пользовательский класс с минимальной функциональностью.
2. Написать функцию для создания объектов пользовательского класса (ввод исходной информации с клавиатуры) и сохранения их в потоке (файле).
3. Написать функцию для чтения и просмотра объектов из потока.
4. Написать функцию для удаления объектов из потока в соответствии с заданием варианта. Для выполнения задания выполнить перегрузку необходимых операций.
5. Написать функцию для добавления объектов в поток в соответствии с заданием варианта. Для выполнения задания выполнить перегрузку необходимых операций.
6. Написать функцию для изменения объектов в потоке в соответствии с заданием варианта. Для выполнения задания выполнить перегрузку необходимых операций.
7. Для вызова функций в основной программе предусмотреть меню.

Создать класс `Pair` (пара чисел). Пара должна быть представлено двумя полями: типа `int` для первого числа и типа `double` для второго. Первое число при выводе на экран должно быть отделено от второго числа двоеточием. Реализовать:

1. вычитание пар чисел
2. добавление константы к паре (увеличивается первое число, если константа целая, второе, если константа вещественная).

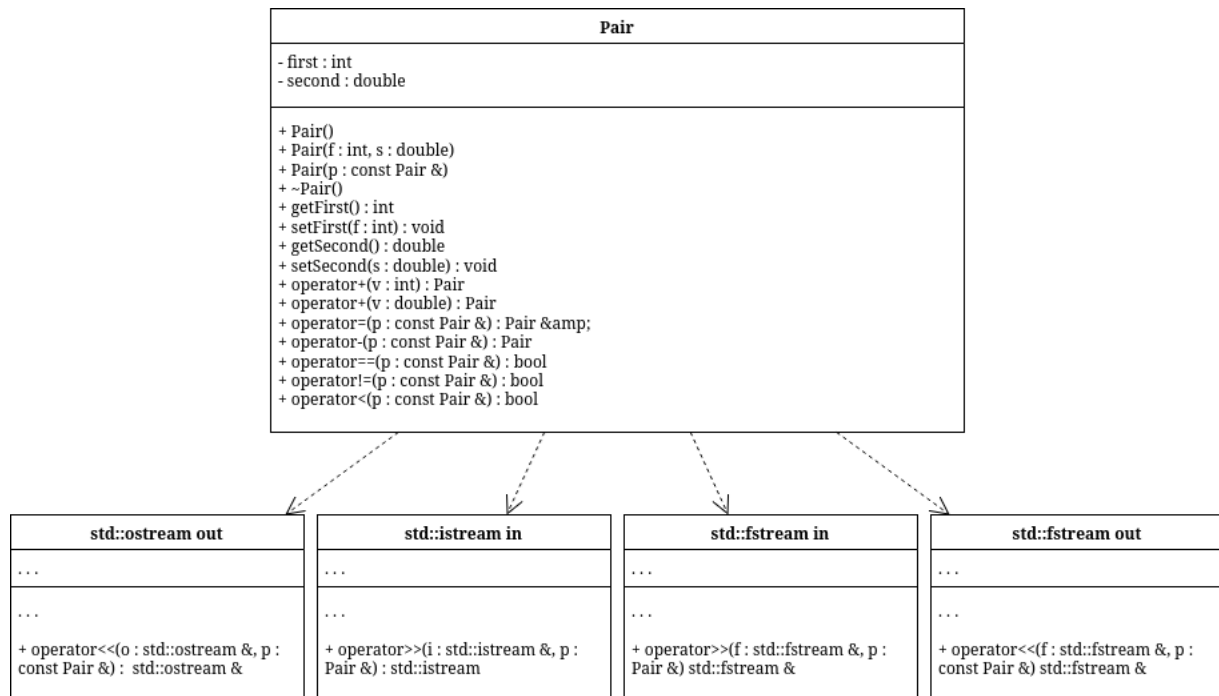
Задание:

1. Удалить все записи меньшие заданного значения.
2. Увеличить все записи с заданным значением на число L.
3. Добавить K записей после элемента с заданным номером.

АНАЛИЗ.

1. Поля Pair приватные, доступ через методы.
2. Формат вывода: first:second.
3. Для удаления и поиска нужны операторы сравнения.
4. Файл обрабатывается через временный файл.

UML-ДИАГРАММА



KOД Pair.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PAIR_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PAIR_H

#include <iosfwd>

class Pair {
    int first;
    double second;

public:
    Pair();
    Pair(int f, double s);
    Pair(const Pair &p);
    ~Pair();

    int getFirst() const;
    double getSecond() const;
    void setFirst(int f);
    void setSecond(double s);

    Pair operator-(const Pair &p) const;
    Pair operator+(int v) const;
    Pair operator+(double v) const;
    Pair &operator=(const Pair &p);

    bool operator==(const Pair &p) const;
    bool operator!=(const Pair &p) const;
    bool operator<(const Pair &p) const;
    bool operator>(const Pair &p) const;

    friend std::ostream &operator<<(std::ostream &out, const Pair &p);
    friend std::istream &operator>>(std::istream &in, Pair &p);
    friend std::fstream &operator<<(std::fstream &out, const Pair &p);
    friend std::fstream &operator>>(std::fstream &in, Pair &p);
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_PAIR_H
```

КОД Pair.cpp

```
#include "Pair.h"
#include <fstream>
#include <iostream>

Pair::Pair() : first(0), second(0.0) {}

Pair::Pair(int f, double s) : first(f), second(s) {}

Pair::Pair(const Pair &p) : first(p.first), second(p.second) {}

Pair::~Pair() = default;

int Pair::getFirst() const {
    return first;
}

double Pair::getSecond() const {
    return second;
}

void Pair::setFirst(int f) {
    first = f;
}

void Pair::setSecond(double s) {
    second = s;
}

Pair Pair::operator-(const Pair &p) const {
    return Pair(first - p.first, second - p.second);
}

Pair Pair::operator+(int v) const {
    return Pair(first + v, second);
}

Pair Pair::operator+(double v) const {
    return Pair(first, second + v);
}

Pair &Pair::operator=(const Pair &p) {
    if (this != &p) {
        first = p.first;
```

```

        second = p.second;
    }
    return *this;
}

bool Pair::operator==(const Pair &p) const {
    return first == p.first && second == p.second;
}

bool Pair::operator!=(const Pair &p) const {
    return !(*this == p);
}

bool Pair::operator<(const Pair &p) const {
    if (first != p.first) {
        return first < p.first;
    }
    return second < p.second;
}

bool Pair::operator>(const Pair &p) const {
    return p < *this;
}

std::ostream &operator<<(std::ostream &out, const Pair &p) {
    out << p.first << ":" << p.second;
    return out;
}

std::istream &operator>>(std::istream &in, Pair &p) {
    in >> p.first >> p.second;
    return in;
}

std::fstream &operator<<(std::fstream &out, const Pair &p) {
    out << p.first << " " << p.second << "\n";
    return out;
}

std::fstream &operator>>(std::fstream &in, Pair &p) {
    in >> p.first >> p.second;
    return in;
}

```

КОД file_work.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FILE_WORK_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FILE_WORK_H

#include "Pair.h"
#include <cstdio>
#include <fstream>
#include <iostream>

int make_file(const std::string &f_name) {
    std::fstream stream(f_name, std::ios::out | std::ios::trunc);
    if (!stream) return -1;

    int n;
    Pair p;
    std::cout << "N? ";
    std::cin >> n;
    for (int i = 0; i < n; i++) {
        std::cin >> p;
        stream << p;
    }
    stream.close();
    return n;
}

int print_file(const std::string &f_name) {
    std::fstream stream(f_name, std::ios::in);
    if (!stream) {
        return -1;
    }
    Pair p;
    int i = 0;
    while (stream >> p) {
        std::cout << p << "\n";
        i++;
    }
    stream.close();
    return i;
}

int del_less(const std::string &f_name, const Pair &key) {
    std::fstream temp("temp", std::ios::out);
    std::fstream stream(f_name, std::ios::in);
    if (!stream) {
```

```

        return -1;
    }
    Pair p;
    int removed = 0;
    while (stream >> p) {
        if (p < key) {
            removed++;
            continue;
        }
        temp << p;
    }
    stream.close();
    temp.close();
    remove(f_name.c_str());
    rename("temp", f_name.c_str());
    return removed;
}

int inc_equal(const std::string &f_name, const Pair &key, double l, bool is_int) {
    std::fstream temp("temp", std::ios::out);
    std::fstream stream(f_name, std::ios::in);
    if (!stream) {
        return -1;
    }
    Pair p;
    int changed = 0;
    while (stream >> p) {
        if (p == key) {
            if (is_int) p = p + static_cast<int>(l);
            else p = p + l;

            changed++;
        }
        temp << p;
    }
    stream.close();
    temp.close();
    remove(f_name.c_str());
    rename("temp", f_name.c_str());
    return changed;
}

int add_after(const std::string &f_name, int n, int k) {
    std::fstream temp("temp", std::ios::out);
    std::fstream stream(f_name, std::ios::in);

```



```

if (!stream) return -1;

Pair p;
int i = 0;
int added = 0;
while (stream >> p) {
    i++;
    temp << p;
    if (i == n) {
        for (int j = 0; j < k; j++) {
            Pair pp;
            std::cin >> pp;
            temp << pp;
            added++;
        }
    }
}
if (i < n) {
    for (int j = 0; j < k; j++) {
        Pair pp;
        std::cin >> pp;
        temp << pp;
        added++;
    }
}
stream.close();
temp.close();
remove(f_name.c_str());
rename("temp", f_name.c_str());
return added;
}

```

```

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FILE_WORK_H

```

КОД main.cpp0

```
#include "Pair.h"
#include "file_work.h"
#include <iostream>
int main() {
    std::string file_name;
    int c = 0;
    do {
        std::cout << "\n1. Make file";
        std::cout << "\n2. Print file";
        std::cout << "\n3. Delete records less than key";
        std::cout << "\n4. Increase records equal to key";
        std::cout << "\n5. Add K records after N";
        std::cout << "\n0. Exit\n";
        std::cin >> c;
        switch (c) {
            case 1: {
                std::cout << "file name? ";
                std::cin >> file_name;
                int k = make_file(file_name);
                if (k < 0) std::cout << "Can't make file\n";
                break;
            }
            case 2: {
                std::cout << "file name? ";
                std::cin >> file_name;
                int k = print_file(file_name);
                if (k == 0) std::cout << "Empty file\n";
                if (k < 0) std::cout << "Can't read file\n";
                break;
            }
            case 3: {
                std::cout << "file name? ";
                std::cin >> file_name;
                Pair key;
                std::cout << "Key pair (int double): ";
                std::cin >> key;
                int k = del_less(file_name, key);
                if (k < 0) std::cout << "Can't read file\n";
                break;
            }
            case 4: {
                std::cout << "file name? ";
                std::cin >> file_name;
```

```

Pair key;
std::cout << "Key pair (int double): ";
std::cin >> key;
std::cout << "Constant type (1-int, 2-double)? ";
int kind = 0;
std::cin >> kind;
int count = 0;
if (kind == 1) {
    int l;
    std::cout << "L? ";
    std::cin >> l;
    count = inc_equal(file_name, key, l, true);
} else {
    double l;
    std::cout << "L? ";
    std::cin >> l;
    count = inc_equal(file_name, key, l, false);
}
if (count < 0) std::cout << "Can't read file\n";

break;
}
case 5: {
    std::cout << "file name? ";
    std::cin >> file_name;
    int n = 0;
    int k = 0;
    std::cout << "N? ";
    std::cin >> n;
    std::cout << "K? ";
    std::cin >> k;
    int added = add_after(file_name, n, k);
    if (added < 0) std::cout << "Can't read file\n";

    break;
}
default:
    break;
}
} while (c != 0);
return 0;
}

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

1. Make file
2. Print file
3. Delete records less than key
4. Increase records equal to key
5. Add K records after N
0. Exit
1
file name?f1
N?5
1 1.1
2 2.2
3 3.3
4 4.4
5 5.5

1. Make file
2. Print file
3. Delete records less than key
4. Increase records equal to key
5. Add K records after N
0. Exit
4
file name?f1
Key pair (int double):1 1.1
Constant type (1-int, 2-double)?1
L?999

1. Make file
2. Print file
3. Delete records less than key
4. Increase records equal to key
5. Add K records after N
0. Exit
5
file name?f1
N?1
K?3
6 6.6
7 7.7
8 8.8

1. Make file
2. Print file

3. Delete records less than key
4. Increase records equal to key
5. Add K records after N
0. Exit

2

file name?f1

1000:1.1

6:6.6

7:7.7

8:8.8

2:2.2

3:3.3

4:4.4

5:5.5

1. Make file
2. Print file
3. Delete records less than key
4. Increase records equal to key
5. Add K records after N
0. Exit

3

file name?f1

Key pair (int double):3 3.3

1. Make file
2. Print file
3. Delete records less than key
4. Increase records equal to key
5. Add K records after N
0. Exit

2

file name?f1

1000:1.1

6:6.6

7:7.7

8:8.8

3:3.3

4:4.4

5:5.5

1. Make file
2. Print file
3. Delete records less than key
4. Increase records equal to key

5. Add K records after N
0. Exit
0

ВОПРОСЫ

1. Что такое поток?

Последовательность данных для ввода/вывода.

2. Какие типы потоков существуют?

Ввод, вывод и двунаправленные; файловые, стандартные, строковые.

3. Какую библиотеку надо подключить при использовании стандартных потоков?

<iostream>

4. Какую библиотеку надо подключить при использовании файловых потоков

<fstream>

5. Какую библиотеку надо подключить при использовании строковых потоков

<sstream>

6. Какая операция используется при выводе в форматированный поток

Оператор <<

7. Какая операция используется при вводе из форматированных потоков?

Оператор >>

8. Какие методы используются при выводе в форматированный поток?

write, put, flush.

9. Какие методы используются при вводе из форматированного потока?

read, get, getline, peek.

10. Какие режимы для открытия файловых потоков существуют?

ios::in, ios::out, ios::app, ios::trunc, ios::ate, ios::binary.

11. Какой режим используется для добавления записей в файл?

ios::app (обычно с ios::out).

12. Какой режим используется в ifstream file("f.txt")?

ios::in.

13. Какой режим используется в fstream file("f.txt")?

ios::in | ios::out.

14. Какой режим используется в ofstream file("f.txt")?

ios::out | ios::trunc.

15. Как открывается поток в режиме `ios::out | ios::app`?

Файл открывается для записи с добавлением в конец.

16. Как открывается поток в режиме `ios::out | ios::trunc`?

Файл очищается и открывается для записи.

17. Как открывается поток в режиме `ios::out | ios::in | ios::trunc`?

Файл создается/очищается и открыт на чтение/запись.

18. Как открыть файл для чтения?

```
ifstream file("f.txt", ios::in);
```

19. Как открыть файл для записи?

```
ofstream file("f.txt", ios::out | ios::trunc);
```

20. Примеры открытия файловых потоков в разных режимах.

```
ifstream("f.txt"); ofstream("f.txt", ios::app); fstream("f.txt", ios::in | ios::out).
```

21. Примеры чтения объектов из потока.

```
stream >> obj; или getline(stream, s).
```

22. Примеры записи объектов в поток.

```
stream << obj; или stream.write(buf, n).
```

23. Алгоритм удаления записей из файла.

1. Чтение 2. запись в temp без удаляемых 3. замена файла.

24. Алгоритм добавления записей в файл.

Для середины: temp, вставка по месту; для конца: `ios::app`.

25. Алгоритм изменения записей в файле.

1. Чтение 2. запись в temp с заменой нужных 3. заменить файл.

